Kinvecta AI Training: Master Guide

Using generative AI well and safely at work: 8 sessions and 2 deep dives

VIDEOS	10 (8 core sessions + 2 deep dives), about 1 h 42 min
PLAYLIST	Kinvecta AI Training: Using Generative AI Well and Safely at Work (YouTube)
AUDIENCE	Everyone (Sessions 1–4), people automating with AI (5–8), builders and reviewers (Deep Dives A–B)
CLASSROOM	90-minute sessions; 120-minute deep dives with hands-on labs

About Kinvecta AI Training

Kinvecta AI Training is a free, 10-video course on using generative AI well and safely at work: an 8-session core series plus two technical deep dives, about 1 hour 42 minutes in total.

- **Watch:** Kinvecta AI Training: Using Generative AI Well and Safely at Work (YouTube playlist)
- **Who it's for:** anyone using AI at work (Sessions 1–4), people automating with AI (Sessions 5–8), and builders and engineers (Deep Dives A and B). Each session is also a 90-minute classroom module for trainers.
- **Stages:** the series follows five stages of AI use: 1 safe everyday use, 2 the API in a container, 3 fixed workflows, 4 tool calling, 5 agents and skills. Everyone starts at Stage 1.
- **How to use this guide:** start with the table below, then open any session for its chapters and key points. Every session ends with one rule of thumb; all ten are in the table below.

Kinvecta: AI training, events and community · kinvecta.ai

At a glance

Ten videos, watched in order from Session 1. Sessions 1–4 are for everyone, Sessions 5–8 for people automating work with AI, and the deep dives for builders.

#	Video	Length	Who it's for	Rule of thumb
1	What Is an LLM? Models, Harnesses, Chatbots, Workflows and Agents	11:12	Everyone	The model thinks. Skills teach it how. Tools let it act. The harness decides which of those it gets.
2	AI Resilience: How to Switch AI Models and Vendors Without Rebuilding	8:41	Everyone	Rent the model. Own the connections. Own the know-how. Own the proof.
3	Safe Prompting and Verification: How to Use AI at Work Without Getting Burned	6:41	Everyone	Right tool for the data. Brief it like a teammate. Check every claim. A person signs.
4	Tokens, Context Windows and How AI Finds Your Data	6:59	Everyone	Send less. Cache what's stable. Fetch with the finder that fits.
5	APIs, Endpoints and Containers: Calling AI From Your Own Code, Safely	6:07	Automators	Call it from code. Keep the endpoint in config. Seal it in a container.
6	How to Build AI Workflows That Don't Break	7:04	Automators	Fix the path. Check every output. A person approves before it ships.
7	AI Tools, MCP and Prompt Injection Explained	7:35	Automators	Read tools run freely, from approved sources. Action tools wait for a person. Everything returned is data.
8	AI Agents, Skills and Running an AI Training Program That Sticks	7:14	Automators and leaders	Write it once, as a skill. Gate what leaves. Measure what changes.
A	Deep Dive: Build an MCP Server from Scratch	20:30	Builders	Few tools, tight schemas. A person confirms every action. Validate your own token. Never pass it on.
B	Deep Dive: DevSecOps for AI — Containers, Scanning, Signing and CI/CD	19:41	Builders and reviewers	Pin everything. Scan before it ships. Sign it, and ship the same digest.

Total: about 1 hour 42 minutes (core series 1:01:33, deep dives 40:11). All videos are in the playlist.

Core series: Sessions 1–8

Each session has a short video, its key points, the chapter list, and six check questions with model answers.

Session 1 · What Is an LLM? Models, Harnesses, Chatbots, Workflows and Agents

Video: in the playlist · 11:12 · For everyone

Key points

- An LLM is a trained engine that predicts and writes text one token at a time. Its weights are frozen after training, so it doesn't learn from your chats; "memory" is the app saving notes and feeding them back in.
- The harness is the software around the model. It decides what the model can see and do, so the same model can be safe in one app and risky in another.
- There are six types of harness: chat assistants, AI built into tools, org platforms, coding and agent tools, open-source agents, and your own software via the API.
- Chatbot, workflow, agent: the person picks each step, a designer fixes the steps in advance, or the AI picks steps at run time. Choose a workflow when the steps are known.

Chapters: 0:00 Welcome · 0:26 What you'll be able to do · 0:52 Why vocabulary comes first · 1:25 What an LLM is · 2:13 How a model is made · 2:47 Six beliefs to correct · 3:29 One request, several models · 4:04 The rule of thumb · 4:23 The parts of an AI system · 5:09 Same model, different harness · 5:50 Six types of harness · 6:39 Reach vs. autonomy · 7:10 Three front doors for the API · 7:48 Chatbot, workflow, agent · 8:41 How far your environment lets you go · 9:15 Exercise · 10:03 Check your understanding · 10:32 Next: AI resilience

Check your understanding

1. In one sentence, what is a large language model?
   - A trained engine that predicts and produces text one token at a time.
2. Why can the same model be safe in one app and risky in another?
   - The harness decides what it can see and do. Security approves the harness, not just the model.
3. Does the model learn from your chats? What is "memory," then?
   - No. The weights are frozen after training. Memory is the harness saving notes and feeding them back in.
4. Name three types of harness and an example of each.
   - Any three of the six: chat assistants, AI built into tools, org platforms or gateways, coding and agent tools, open-source agents, your own software via the API.
5. Who picks each step in a chatbot, a workflow and an agent?
   - The person; a designer, in advance; the AI, at run time.
6. When would you choose a workflow over an agent?
   - When the steps are known in advance. It's more predictable, and easier to authorize and test.

Session 2 · AI Resilience: How to Switch AI Models and Vendors Without Rebuilding

Video: in the playlist · 8:41 · For everyone

Key points

- Being agnostic means you can swap the model, or the whole vendor, without rebuilding your work. There are four kinds of switch: model version, vendor, endpoint and harness.
- Seven foundations make a switch easy: approved MCP tools, containers, scanning, skills, code packages, a multi-vendor API layer and stored evals.
- Keep the model name and endpoint in configuration, with the version pinned, so a switch is a one-line change.
- Stored evals are your proof. They catch drift, both when a new model answers differently and when the business or data changes under an unchanged prompt.

Chapters: 0:00 Welcome · 0:27 What you'll be able to do · 0:48 Why resilience matters · 1:15 What "agnostic" means · 1:51 Four kinds of switch · 2:24 Four beliefs to correct · 3:00 Seven foundations, one loop · 3:50 Seven things you own · 4:29 The one-line model switch · 5:06 One skill, many harnesses · 5:40 Evals catch drift · 6:17 The rule of thumb · 6:33 The switching playbook · 7:05 Exercise: lock-in audit · 7:43 Check your understanding · 8:11 Next: safe prompting and verification

Check your understanding

1. In one sentence, what makes an organization agnostic?
   - It can swap the model, or the whole vendor, without rebuilding its work.
2. Name four of the seven foundations.
   - Any four of: approved MCP tools, containers, scanning, skills, code packages, a multi-vendor API layer, stored evals.
3. Why do stored evals matter most when you switch?
   - They're the only evidence the new model works as well as the old one. Without them, a switch is a guess.
4. What is the difference between model drift and use-case drift?
   - Model drift: a new model version answers differently. Use-case drift: the business, data or users change while the prompt stays the same.
5. Where should the model name and endpoint live?
   - In configuration, with the version pinned, not hard-coded.
6. Why test a skill in each harness if the format is shared?
   - Scripts, file paths and tool names can differ between harnesses, so a skill can fail even though the format is shared.

Session 3 · Safe Prompting and Verification: How to Use AI at Work Without Getting Burned

Video: in the playlist · 6:41 · For everyone

Key points

- The one rule: data only goes into a tool approved for that data's sensitivity level. If you paste the wrong thing, stop, report it right away and record what was shared.
- Prompt like a briefing: role, task, audience and format, plus the source material. Pasting the source keeps the model working from your material, not its memory.
- Fluent is not correct. Check every claim: sources, numbers, names and dates, quotes, what's missing, and tone and bias.
- AI drafts, a person signs. A prompt that works becomes a saved, reviewed and tested instruction in the team library.

Chapters: 0:00 Welcome · 0:26 What you'll be able to do · 0:45 Why habits matter · 1:11 The one rule · 1:47 Match the data to the tool · 2:14 If you paste the wrong thing · 2:37 AI drafts, a person signs · 3:02 Prompt like a briefing · 3:42 Check every claim · 4:14 The verification checklist · 4:33 Save what works · 5:07 The rule of thumb · 5:24 Exercise: brief it, check it, save it · 5:53 Check your understanding · 6:19 Next: tokens, context and how AI finds your data

Check your understanding

1. What is the one rule?
   - Data only goes into a tool approved for that data's sensitivity level.
2. What do you do right after pasting something you shouldn't have?
   - Stop, report it to your security or privacy contact right away, record what was shared, then follow the incident process.
3. What are the parts of a briefing prompt?
   - Role, task, audience and format, plus the source material.
4. Name four things to check before using a draft.
   - Any four of: sources, numbers, names and dates, quotes, what's missing, tone and bias.
5. Who is accountable for AI-assisted work?
   - A named person. AI drafts, a person signs.
6. What makes a prompt ready for the team library?
   - A header (name, purpose, owner, inputs, data level), a second person's review, and passing test cases.

Session 4 · Tokens, Context Windows and How AI Finds Your Data

Video: in the playlist · 6:59 · For everyone

Key points

- A token is roughly four characters, about three quarters of an English word. Codes and numbers use more, and every vendor counts differently.

- The context window is one fixed budget shared by everything: instructions, tools, history, documents, the question and the answer. What you send costs more than what you get back.
- Long chats get more expensive every turn because the whole conversation is re-sent. Caching makes the stable prefix cheap; start a new chat when the topic changes.
- The model only sees what finders return. Match the finder to the data, and remember a search index is a copy of your data that needs the same access rules.

Chapters: 0:00 Welcome: what can the model see? · 0:30 What you'll be able to do · 0:49 Three complaints, one root cause · 1:16 What a token is · 1:48 Types of tokens · 2:19 The context window budget · 2:52 Why long chats cost more · 3:19 Six reasons tokens matter · 3:46 How AI finds your data · 4:23 Six ways AI fetches data · 4:58 Match the finder to the data · 5:25 The rule of thumb · 5:37 Exercise: count it, then find it · 6:14 Check your understanding · 6:37 Next: APIs, endpoints and containers

Check your understanding

1. Roughly how much text is one token?
   - About four characters, or three quarters of an English word. Codes and numbers use more.
2. Why do long chats cost more each turn?
   - Every turn re-sends the system prompt and the whole history.
3. What shares the context window?
   - Everything sent, plus the answer: instructions, tools, history, documents, the question and the output.
4. What does caching make cheaper?
   - A stable prefix that repeats, such as instructions, tool definitions and reference documents.
5. Name three ways AI fetches data.
   - Any three of: everything in context, semantic search, keyword search, agentic search, structured queries, multimodal retrieval.
6. Why is a search index a security question?
   - An index is a copy of the data, often stored elsewhere, so it needs the same access rules and approvals as the source.

Session 5 · APIs, Endpoints and Containers: Calling AI From Your Own Code, Safely

Video: in the playlist · 6:07 · For people automating with AI

Key points

- When your code calls the model through the API, your software becomes the harness. The six jobs the chat app did now belong to your team: instructions, data handling, logging, keys and network, review, and cost limits.
- There are three front doors to the same model: the vendor's own API, a cloud provider's hosted version, and your organization's approved platform. Choose the door by its terms and approvals first, then the model.
- Four habits make a switch a one-line change: endpoint in config, pinned model version, one request shape, and an allow-list for that one endpoint.

- Every harness runs sealed in a container with one way out. Every run leaves a record: secrets at run time, a pinned model, an audit log, and rate and cost limits.

Chapters: 0:00 Welcome: who owns the controls? · 0:27 What you'll be able to do · 0:46 Three problems that start when code calls the model · 1:14 Through the API, your code becomes the harness · 1:43 Six jobs your code now owns · 2:10 Three front doors to the same model · 2:40 Four habits: keep the endpoint in config · 2:57 Sealed in a container, one way out · 3:30 The four run controls · 3:56 What you can do today · 4:16 The rule of thumb · 4:30 Exercise: read it, then box it · 5:02 Check your understanding · 5:29 Next: building workflows · 5:46 For facilitators: the Stage 2 sandbox

Check your understanding

1. When your code calls the model instead of the chat app, what does your team now have to handle?
   - The six jobs the chat app did: instructions (saved prompts), data handling, logging, keys and network, human review, and cost limits.
2. Name the three kinds of endpoint.
   - The vendor's own API, a cloud provider's hosted version, and an approved platform or gateway.
3. Which do you choose first, the door or the model, and why?
   - The door. The model behind every door is the same; what changes is the contract: terms, data retention and approval.
4. Why keep the endpoint in config?
   - So switching is a one-line change, and the network allow-list can match it.
5. What can a container reach?
   - Only the approved model endpoint and any approved tool servers. Everything else, including the host's own files, is blocked.
6. Name the four run controls.
   - Secrets injected at run time from a secrets store, a pinned model version, an audit log of every run, and rate and cost limits.

Session 6 · How to Build AI Workflows That Don't Break: Fixed Steps, Checks and Human Review

Video: in the playlist · 7:04 · For people automating with AI

Key points

- In a workflow, a designer fixes the path before it runs. In a chatbot the person picks each step; in an agent the AI does.
- Chat plus an API is enough for Stage 3: working, containerized automation. You don't need agents to start.
- A workflow has six fixed steps, including one check and one person: trigger, pull inputs, saved prompt, validate, human review, deliver. Every step writes to the audit log.
- Bad output is flagged, never passed on. A new prompt version is a new release, tested against known-good cases and one tricky case before it goes live.

Chapters: 0:00 Welcome: three Mondays, one fixed path · 0:34 What you'll be able to do · 0:56 Why repeat work needs a fixed path · 1:23 Chatbot, workflow or agent: who picks each step? · 2:01 What you can build today · 2:32 Score your first workflow · 2:56 Anatomy of a workflow · 3:36 Check every output · 4:06 Run it safely: sealed in, logged, capped · 4:32 Test it, then version it · 5:01 Four numbers that prove it's working · 5:16 The rule of thumb · 5:31 Exercise: design it, then break it · 6:12 Check your understanding · 6:42 Next: tools, MCP and prompt injection

Check your understanding

1. Who decides the steps in a workflow?
   - A designer, in advance. The script follows the same path every run.
2. What can a team build with only chat and an API?
   - Scripted or low-code workflows that call the model through the API, with MCP servers as fixed steps. That's Stage 3.
3. Name three signs of a good first workflow.
   - Any three of: repeats weekly, same shape every time, a known-good output, already reviewed, approved data level, time you can measure.
4. What happens when a validation check fails?
   - The run is flagged and routed to a person. Nothing is delivered.
5. Where should a workflow's prompts live?
   - As saved, versioned, reviewed files in a shared approved location, not typed into each flow.
6. A report in the inputs says "ignore your rules." Which two parts of the workflow stop that from shipping?
   - The validation check, which flags unexpected links, recipients or actions, and the human review gate. The tricky case in the test set catches it before release, too.

Session 7 · AI Tools, MCP and Prompt Injection Explained: Letting AI Act Without Getting Hijacked

Video: in the playlist · 7:35 · For people automating with AI

Key points

- The model only asks for a tool; the harness decides and runs it. Reads from approved sources run freely, actions wait for a person, and every result comes back as data, not instructions.
- MCP (the Model Context Protocol) is the open standard for plugging tools into any harness. Build one server and a chat app, a workflow and an agent can all use it.
- Prompt injection, instructions hidden in content the AI reads, is ranked the number-one risk for AI applications by OWASP. An attack needs private data, untrusted content and a way out; remove any one and it fails.
- No single defense stops it; layers do. Tools are software: same review, testing, scanning and release pipeline as any code.

Chapters: 0:00 Welcome: the model asks, the harness decides · 0:26 What you'll be able to do · 0:47 Why tools raise the stakes · 1:11 Tool calling · 1:44 The parts of an AI system · 2:07 MCP: one tool server, many harnesses · 2:38 How an MCP call works · 3:10 Three rules for building a tool · 3:28 Prompt injection · 4:16

The three ingredients of an attack · 4:41 Layered defenses · 5:24 Tools are software · 5:47 The rule of thumb · 6:02 Exercise: trace it, then gate it · 6:47 Check your understanding · 7:13 Next: agents, skills and running the program

Check your understanding

1. Who runs a tool: the model or the harness?
   - The harness, meaning your code. The model only asks.
2. Why can one MCP server serve a chat app, a workflow and an agent?
   - MCP is an open standard, so every harness that speaks it can use the same server and its tools, resources and prompt templates.
3. Why do tool descriptions need review?
   - Names and descriptions are instructions the model reads, so a careless or malicious one changes its behavior.
4. What is indirect prompt injection?
   - Instructions planted in content the AI reads later, such as a document, web page, email or record.
5. Which tools need a person's approval?
   - Any tool that sends, changes or deletes something.
6. Name three defenses against prompt injection.
   - Any three of: treat content as data, least access, approval for actions, output validation, a container limited to approved endpoints, line-by-line review of outside skills.

Session 8 · AI Agents, Skills and Running an AI Training Program That Sticks

Video: in the playlist · 7:14 · For people automating with AI, and program leads

Key points

- A skill is like an onboarding guide the AI loads only when needed. Only names and descriptions load up front, so dozens can be installed cheaply. A vague description means a skill that never fires.
- Treat every skill like code: read it, scan it, pin the version, install from an internal registry, and decide who may install.
- A skill brings the method, tools bring facts and actions, and the model fills the gaps. An agent runs in a container, logs every step, and holds at the gate until a person approves anything that leaves.
- Everyone starts at Stage 1, and each stage adds one new risk. Training worked when the work changed: take baselines first, then measure time saved, errors caught, reuse and workflows in production.

Chapters: 0:00 Welcome: putting the pieces together · 0:26 What you'll be able to do · 0:47 Three ways AI programs stall · 1:16 How a skill loads in layers · 1:45 Treat every skill like code · 2:11 Skill + tools + model: one repeatable job · 2:51 Agents with guardrails · 3:25 One shared prompt and skill library · 3:50 Five

stages, one new risk each · 4:32 Measuring what changed · 5:08 The rule of thumb · 5:21 Exercise: package it, then plan it · 6:04 Check your understanding · 6:31 Series complete: your next step

Check your understanding

1. What loads before a skill is triggered?
   - Only each skill's name and description, about 80 tokens each.
2. Which stage does everyone complete, and where do analysts stop?
   - Everyone completes Stage 1. Analysts stop at Stage 3.
3. What do a skill, tools and a model each add?
   - The skill adds the method, the tools add facts, actions and checks, and the model adds judgment in the gaps.
4. Where must a person approve an agent's work?
   - Before any action leaves the system: sending, writing back or changing a record.
5. What makes a prompt ready for the library?
   - A plain-text file with a header, a second person's review, versioning, and test cases it passes.
6. Name two measures that show training worked.
   - Any two of: time saved per task, errors caught in review, library reuse, workflows in production, security findings.

Deep dives

Two hands-on, DevSecOps-minded deep dives for builders and reviewers. They follow Session 7 and pair with each other: A builds the server, B boxes, scans, signs and ships it.

Deep Dive A · Build an MCP Server from Scratch: Current Spec, Python & TypeScript, Security Built In

Video: in the playlist · 20:30 · For builders

Key points

- The MCP spec changed on 28 July 2026: no start-up handshake or sessions, and every request carries its own protocol version. Both official SDKs moved to version 2 with renamed APIs, so older tutorials are legacy.
- Three roles (host, client, server) and four layers with trust boundaries. Each layer enforces its own rules, and results always travel back as data.
- Tools, resources and prompts are controlled by the model, the app and the user. In a tool definition, the model reads the name and description, the SDK enforces the schema, and annotations are only hints.
- Actions ask a person first via a multi-round-trip request. Always handle no and cancel, or the server loops.
- stdio for one person, Streamable HTTP behind a gateway for teams, containers with an egress allow-list in production. Remote sign-in is OAuth 2.1; never pass the user's token on.
- Test like a reviewer (Inspector, the decline path, a tool-list snapshot gate in CI) and treat it as a DevSecOps loop: design, build, test, secure, ship, operate.

Chapters: 0:00 Welcome: build a server you can trust · 0:37 What you'll be able to do · 1:11 Why build it carefully · 2:05 Host, client, server · 2:59 The big picture: four layers, three trust boundaries · 3:52 Tools, resources and prompts · 4:34 Anatomy of a tool definition · 5:12 One request on the wire today · 6:03 Set up on the current SDKs · 6:54 Python: a read-only tool in 16 lines · 8:04 Python: an action that asks a person first · 9:01 How the server asks: multi-round-trip requests · 9:59 TypeScript: the decline fix · 10:51 stdio vs Streamable HTTP · 12:01 Where an MCP server runs · 12:50 Build, inspect, break, fix · 13:42 Connect it to a host · 14:31 OAuth 2.1, and why no token passthrough · 15:44 Threats and server-side controls · 16:58 DevSecOps for an MCP server · 17:54 The rule of thumb · 18:10 Hands-on lab · 19:23 Check your understanding · 20:11 Next: Deep Dive B

Check your understanding

1. With no start-up handshake, how does a server learn the client's protocol version?
   - From the metadata on every request, which carries the protocol version and the client's capabilities. Clients may also call server/discover first.
2. Your stdio server works by hand, but the host says it failed. Why?
   - Something wrote to standard out (a print or console.log), corrupting the protocol stream. Log to standard error, and check absolute paths and environment variables in the host config.

3. destructiveHint is false. Can the host auto-approve the tool on that basis?
   - No. Annotations are untrusted hints and enforce nothing. Enforcement is the server's authorization, scopes and confirmation.
4. Why must a server never forward the token it received?
   - That's token passthrough, which the spec forbids. The token is meant for the MCP server, so forwarding it bypasses downstream controls, breaks the audit trail and turns the server into an exfiltration proxy. Use the server's own credential.
5. How does a server get a person's confirmation in the middle of a call?
   - It returns an input-required result with a confirmation form. The client asks the person and retries the call with the answer and a new request ID.
6. Name two controls against a malicious or changed server.
   - Any two of: pin exact versions and digests, install from verified publishers or an internal mirror, snapshot and diff the tool list in CI, restrict outbound network access, least-privilege tokens.

Deep Dive B · DevSecOps for AI: Containers, Image Scanning, Signing and the CI/CD Pipeline

Video: in the playlist · 19:41 · For builders and reviewers

Key points

- DevSecOps means security in every stage, not one gate at the end: shift left, and keep watching right.
- An image is a sealed box; its digest is its true identity, while a tag can move. The same image runs on a laptop, a server, serverless containers, Kubernetes or the edge; who patches what changes, but your image, code, keys and network rules stay yours.
- Harden the image (minimal base pinned by digest, multi-stage build, non-root) and keep secrets out of images and repos: inject them at run time from a secrets manager.
- Use several scanners, then triage by real risk: actively exploited (KEV) first, then likely to be exploited (EPSS), then fixable critical and high.
- Sign and attest the digest, verify it at admission, and promote the same digest from staging to production. Pin CI actions to full commit SHAs, because your scanner is a dependency too.
- Firewalls: default deny in both directions, layer by layer. A Docker-published port bypasses ufw, and AI workloads should reach only the LLM API and approved backends through an egress allow-list.

Chapters: 0:00 Welcome: one box, trusted everywhere · 0:33 What you'll be able to do · 0:56 Three problems a good pipeline solves · 1:33 DevSecOps: security in every stage · 2:19 Image, container, registry and digest · 3:06 Where containers run · 4:05 Hardening the image · 4:52 Reading a hardened Dockerfile · 6:00 Where secrets live · 6:58 A scanner at every station · 7:47 Scanning tools by category · 8:57 Triage: fix what's exploited first · 10:01 Sign, attest, verify · 10:52 Signing, registry, admission and runtime tools · 11:51 The CI/CD line and its gates · 12:41 A GitHub Actions pipeline, annotated · 13:44 When the scanner is the attack · 14:31 Firewalls: default deny, both directions · 15:27 Network layers and the tools for each · 16:42 AI-

specific gates · 17:23 The rule of thumb · 17:38 Exercise: find the problems, then triage · 18:27 Check your understanding · 19:22 Wrap-up: build it, box it, ship it

Check your understanding

1. Why sign and deploy by digest instead of tag?
   - A tag is a movable label; a digest is a hash of the exact bytes. It guarantees that what was scanned and approved is what runs.
2. A key was added in one step and deleted in the next. Is the image safe to publish?
   - No. Each step is a permanent layer, so the key is still there. Rotate it now, rebuild with run-time injection or build-secret mounts, and add secret scanning.
3. A scan shows 40 HIGH findings. What do you fix first?
   - KEV first, then high EPSS, then fixable critical and high, focusing on what actually runs. Document the rest with VEX. Often a newer minimal base image is the fastest fix.
4. Scanning versus signing: why do you need both?
   - Scanning checks the contents for known problems; signing proves identity and origin and that nothing was altered. Without signing someone can swap the box; without scanning you faithfully ship something vulnerable.
5. ufw denies incoming traffic, but a port published with docker run -p is still reachable. Why, and what are two fixes?
   - Docker publishes ports through its own NAT rules, which bypass ufw. Bind the port to localhost behind a reverse proxy, or put rules in the DOCKER-USER chain. Treat the cloud security group as the real perimeter and test from outside with nmap.
6. A scanner's GitHub Action was hijacked. Which two controls limit the damage?
   - Pin actions to full commit SHAs (and scanner images by digest), and keep CI least-privilege with short-lived OIDC credentials, minimal permissions and fast secret rotation.